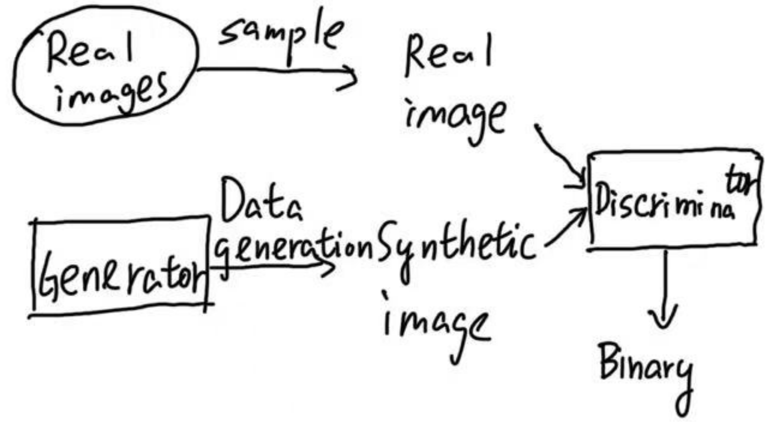
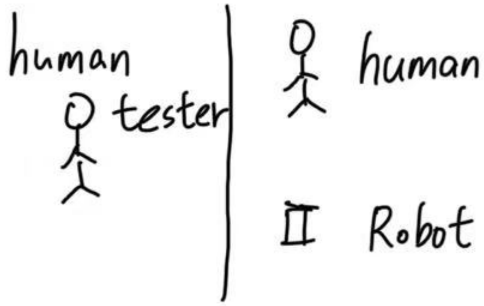
Generative adversarial network (GAN)



1 synthetic

0 Real

Turing test



Discriminator

Binary image-classification model

1 synthetic

0 Real

Image → D → Binary label

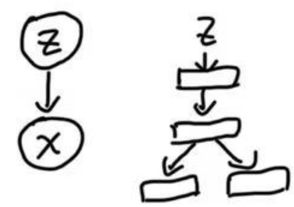
$\begin{cases} \text{synthetic} \\ \text{Real} \end{cases}$

ResNet

Vision Transformer

Generator

the same as VAE



VAE: maximize date likelihood

GAN: Turing test

$\hat{z} \sim N(0, I)$

$\hat{x} \sim P(x \mid \hat{z})$

Training dataset
for Discriminator

Positive examples $\underline{P(G)}$
(synthetic image1, 1)
(synthetic image2, 1) ....

Negative examples $\underline{N}$
(Real image 1, 0)
(Real image 2, 0)
⋮

z → x

$\hat{z} \sim N(0, I)$

$\hat{x} \sim \underbrace{f(\hat{z}; G)}_{\text{mean}} \quad \underbrace{g(\hat{z}; G)}_{\text{covariance}}$

$\hat{x}(\hat{z}, \underline{G})$

$\max_{G} \min_{D} \underbrace{L}_{\text{Cross-entropy}} (\underbrace{D}_{\text{parameters}}, \underbrace{P(G), N}_{\text{Training data}})$

optimization (train the GAN model)

while not converged

① fix G, optimize D $\min_{D} L(D, \underline{P(G)}, \underline{N})$ Binary classification

② fix D, optimize G $\max_{G} L(D, P(G), N)$

$G \rightarrow$ synthetic images $\xrightarrow{D} L(D, P(G), N)$

Text-to-image Generation $\underline{\hat{z}} \sim N(0, I)$ $\underline{\text{Vanilla}}$ GAN

Conditional generation

Conditional GAN

$\underline{\text{Text}}$ → [Text encoder] → Text vector $\underline{\text{representation}}$ → [Generator] → Synthetic image

(t) $(\hat{z}) \sim N(0, I)$

random $\hat{z}$ [], $\underline{t}$ [] → [] → $f(\hat{z}, t)$ mean ; → $g(\hat{z}, t)$ covariance

$\hat{x} \sim P(x \mid f(\hat{z}, t), \operatorname{diag}(g(\hat{z}, t)))$

Discriminator

image → [D] → Binary label   Image classification

text, image → [D] → Binary label   Multi-modality classification
Textual-visual classification

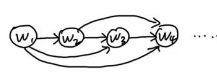
Text → [Text encoder] → t
Image → [Image encoder] → i
→ []t []i → MLP → label

VAE   z → x

GAN

Large language models

$\underbrace{P(w_1 w_2 w_3 \cdots w_N)}_{\text{text}}$



$P(w_{\underline{1:N}}) = p(w_1) \prod_{k=2}^{N} \underline{p(w_k | w_{1:k-1})}$

Transformer

Recurrent network

Word embedding

apple → t_{apple}

desk → t_{desk}

$\underline{\text{word}}$ $\underline{\text{token}}$

50,000

I like apple because apple is ...
3rd   5th

Lookup table